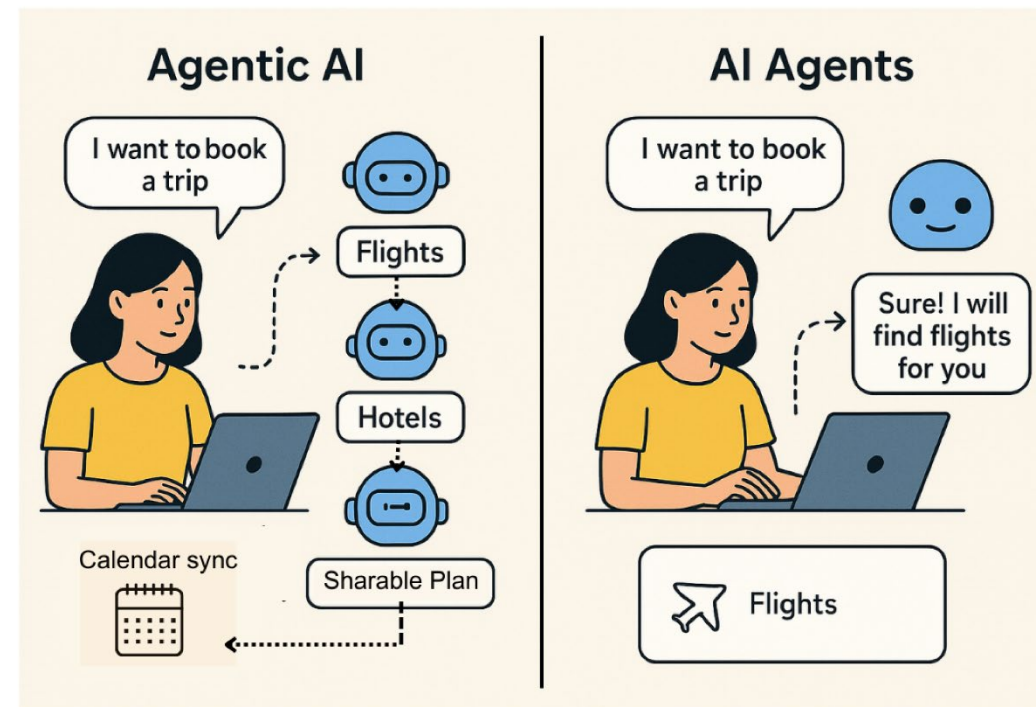


AI Agent, Agentic AI, and Evolutionary Computation



<https://www.mdpi.com/1999-5903/17/9/404>

What is an AI Agent - The Autonomous "Doers"

- An AI Agent is a software system that perceives its environment and takes actions to achieve a specific, pre-defined goal. It acts *on behalf* of a user.
- Key Characteristics:
 - Reactive: Typically waits for a trigger or specific input to execute a workflow.
 - Task-Specific: Designed for a narrow scope (e.g., retrieving data, navigating a map, playing chess).
 - Tool Use: Can utilize external tools (calculators, APIs) but usually follows a rigid "If-Then" logic or single-step reasoning.
 - The Loop: Perception → Decision → Action.
- Examples:
 - Siri / Alexa: Executes specific commands ("Set a timer").
 - Recommendation Algorithms: Acts to maximize click-through rates.
 - Trading Bots: Buys/sells stock based on set thresholds.

What is Agentic AI? The Adaptive "Planners"

- Agentic AI refers to systems possessing a high degree of **agency**—the ability to act proactively, reason through complex goals, and adapt strategies without continuous human intervention.
- The Paradigm Shift: Moving from "Tools that wait for commands" → to "Teammates that manage objectives."
- **Key Differentiators:**
 - **Goal-Oriented (Not Task-Oriented):** You give it an abstract objective (e.g., "Plan a marketing campaign"), and *it* figures out the necessary tasks.
 - **Multi-Step Reasoning:** Capable of creating sub-goals, self-reflection ("Did this work?"), and error correction.
 - **Cross-Domain Autonomy:** Can span multiple applications (e.g., browsing the web, writing code, and emailing files) in a single workflow.

Feature	AI Agent (The "Doer")	Agentic AI (The "Thinker")
Primary Driver	Trigger-based: Waits for a specific command to start.	Goal-based: Pursues a high-level objective proactively.
Scope of Work	Narrow: Specialized in one domain (e.g., retrieving a weather forecast).	Broad: Can span multiple domains (e.g., "Plan a vacation" involves weather, flights, hotels, and calendar).
Process	Linear: Input → Processing → Output.	Cyclical: Plan → Act → Observe → Reflect → Adjust.
Handling Failure	Brittle: If an API fails or input is wrong, it stops or errors out.	Resilient: Self-corrects (e.g., "That link is broken, I will try a Google search instead").
Human Role	Pilot: The human steers every step.	Manager: The human sets the goal and reviews the final result.
Example 1	Book a flight to NYC.	Get me to NYC for my meeting. (Checks calendar, finds flights, books hotel, arranges Uber, and updates the team)
Example 2	“Find me a recipe for chocolate cake.”	“Find a chocolate cake recipe, check if I have the ingredients in my smart fridge, and add missing items to my grocery cart.”

How Evolutionary Computation (EC) is applied to optimize AI Agents and Agentic AI systems

The core concept is to treat the agent's components as "**DNA/Gene**" that can be mutated and selected for survival.

- Evolving the Instructions to the agent (Prompt Optimization)
- Evolving the Skills (Tool Optimization)
- Evolving the Structure (Architecture Search)

Evolutionary Loop for finding best agents

1. Initialize: Create a batch of agents (random prompts/tools).
2. Evaluate: Test them on a goal.
3. Select: Keep the top 10% for example (static extinctive selection)
4. Mutate: Use an LLM to mix and produce improved agent. ← No need math. AI LLM is the mutator
5. Repeat: Until the agent is perfect.

Evolving the Instructions (Prompt Optimization)

- The “DNA/Gene”: The System Prompt (e.g., "You are a helpful assistant...").
- Method: The system generates variations of a prompt, tests them on a task, and keeps the high-performing ones.
- The Mutation: Using an LLM to rewrite the best prompts to be clearer or more effective.
- Benefit: Discovers phrasing that humans wouldn't guess but that machines understand perfectly.

What is a System Prompt? In the context of LLMs and AI Agents, it is the initial, hidden set of instructions given to the AI *before* the user interacts with it. It serves as so-called the "God Mode" configuration that defines who the AI is, how it should behave, and what rules it must follow.

Evolving the Skills (Tool Optimization)

- The “DNA/Gene”: The code or Python functions the agent uses
- Method: When an agent performs poor or fails a task, the system uses the error message to "mutate" (rewrite) the code.
- The Selection: Only code that runs successfully and solves the problem is saved to the agent's "Skill Library".
- Benefit: The agent builds its own toolbox without human coding.

Evolving the Structure (Architecture Search)

- The “DNA/Gene”: The agent workflow or agent team composition
- Method: The system randomly adds or removes specific agents or changes the communication flow.
- The Selection: Architectures that produce higher quality outputs with fewer steps survive.
- Benefit: Optimizes complex multi-agent teams for specific goals.

(황승진 스탠포드 경영대학원 석좌교수)

- <https://youtu.be/eQUzEBKLIjs?si=RUdHqUGSHDb4BMsp>

